भारतीय प्रौद्योगिकी संस्थान धारवाड

Indian Institute of Technology Dharwad

INDIAN INSTITUTE OF TECHNOLOGY DHARWAD

Department of Computer Science and Engineering

CS601T : Deep Learning

Programming Assignment - I

Perceptron Learning for Classification and Regression

Assignment Number : 1

Course : CS601T - Deep Learning

Assignment Title : Classification and Regression using Perceptron

Group Number : 24

Group Members

MC23BT011 Sadguru Sai

MC23BT022 Kailas Santhosh

MC23BT030 Shaikh Aariz

Submission Details

Submitted To : Prof. Dileep A. D

Department : Computer Science and Engineering

Institute : Indian Institute of Technology Dharwad

Date of Submission : 23 August 2026

Academic Year 2026–2027

Contents

List of Figures

List of Tables

1. Introduction

Artificial Intelligence (AI) and Machine Learning (ML) have become essential technologies for solving complex real-world problems. At the core of many modern neural network architectures lies the perceptron, the earliest computational model capable of learning a decision boundary directly from training data.

The perceptron, proposed by Frank Rosenblatt in 1958, forms the fundamental building block of Artificial Neural Networks (ANNs). Although simple in structure, it introduces the key concepts of weighted inputs, activation functions, error computation, and iterative parameter optimization using learning algorithms.

This programming assignment focuses on implementing a single-layer perceptron completely from scratch without using any machine learning frameworks. The objective is to understand every mathematical and computational step involved in training neural networks.

The implemented perceptron is applied to both classification and regression problems. For classification, Logistic (Sigmoid) and Hyperbolic Tangent (Tanh) activation functions are investigated on both linearly separable and non-linearly separable datasets. Since each dataset contains three classes, the One-Versus-One (OVO) strategy is employed to perform multiclass classification.

For regression, a perceptron with Linear activation is trained on univariate and bivariate datasets to predict continuous target values. Model performance is evaluated using quantitative metrics together with graphical visualizations such as learning curves, decision boundaries, regression plots and scatter plots.

This report presents the implementation methodology, experimental setup, obtained results, detailed performance analysis and important observations drawn from the conducted experiments.

2. Objectives

The primary objectives of this assignment are listed below.

- Implement a single-layer perceptron completely from scratch using Python.
- Understand the mathematical formulation of perceptron learning.
- Implement Gradient Descent for iterative weight optimization.
- Perform multiclass classification using the One-Versus-One strategy.

- Compare Logistic and Hyperbolic Tangent activation functions.
- Implement Linear activation for regression problems.
- Evaluate model performance using suitable quantitative metrics.
- Visualize the learning behaviour using decision boundaries and learning curves.
- Analyse the strengths and limitations of a single-layer perceptron.

3. Assignment Tasks

The programming assignment consists of two major parts: Classification and Regression. In the classification task, a single-layer perceptron is trained using two different activation functions and evaluated on both linearly separable and non-linearly separable datasets. In the regression task, the perceptron is trained using linear activation to predict continuous-valued outputs.

3.1 Classification

The classification task consists of the following objectives.

- Train a perceptron using the Logistic (Sigmoid) activation function.
- Train a perceptron using the Hyperbolic Tangent (Tanh) activation function.
- Perform multiclass classification using the One-Versus-One (OVO) strategy.
- Evaluate the classifier on a linearly separable dataset.
- Evaluate the classifier on a non-linearly separable dataset.
- Plot learning curves showing Average Error versus Epochs.
- Visualize the learned pairwise decision boundaries.
- Generate the final combined three-class decision regions.

3.2 Regression

The regression task consists of the following objectives.

- Implement a perceptron using Linear activation.

- Perform Univariate Regression.
- Perform Bivariate Regression.
- Evaluate the regression model using RMSE and Percentage RMSE.
- Plot Average Error versus Epochs.
- Compare target outputs with predicted outputs.
- Analyse the Target versus Predicted Scatter plots.

4. Learning Outcomes

After completing this programming assignment, the following learning outcomes were achieved.

- Developed a complete understanding of the perceptron learning algorithm.
- Understood Gradient Descent based parameter optimization.
- Learned the mathematical formulation of Logistic, Hyperbolic Tangent and Linear activation functions.
- Understood the implementation of One-Versus-One multiclass classification.
- Learned the importance of feature scaling and model convergence.
- Gained practical experience in implementing machine learning algorithms without using external ML libraries.
- Understood the strengths of perceptrons for linearly separable problems.
- Understood the limitations of single-layer neural networks on non-linearly separable datasets.
- Learned the evaluation of classification and regression models using quantitative metrics and graphical analysis.

5. Experimental Setup

The perceptron learning algorithm was implemented entirely from scratch in Python. No machine learning libraries such as TensorFlow, PyTorch, Scikit-Learn or Keras were used during

implementation. Numerical computations were carried out using NumPy, while Matplotlib was used exclusively for plotting graphs and visualizing the experimental results.

The model parameters were optimized using the Gradient Descent algorithm. Separate perceptrons were trained depending upon the activation function and the learning task.

5.1 Development Environment

Table 5.1: Software Environment Used for Implementation

Component	Description
Programming Language	Python 3.x
Development Environment	Jupyter Notebook
Numerical Library	NumPy
Visualization Library	Matplotlib
Machine Learning Libraries	None
Implementation	From Scratch
Optimization Method	Batch Gradient Descent

5.2 Dataset Description

Four datasets were provided for this assignment. Two datasets were used for classification and two datasets were used for regression.

Table 5.2: Summary of Datasets Used

Task	Dataset	Input Features	Output
Classification	Linearly Separable	2	3 Classes
Classification	Non-Linearly Separable	2	3 Classes
Regression	Univariate	1	Continuous
Regression	Bivariate	2	Continuous

All datasets were divided into training and testing subsets following the 70%–30% split specified in the assignment.

5.3 Training Configuration

The perceptron models were trained using Batch Gradient Descent. Different activation functions were selected according to the learning problem.

Table 5.3: Training Configuration

Parameter	Configuration
Training Data	70%
Testing Data	30%
Optimization Algorithm	Batch Gradient Descent
Classification Activation Functions	Logistic, Tanh
Regression Activation Function	Linear
Multiclass Strategy	One-Versus-One (OVO)
Feature Scaling	Standardization (where applicable)
Stopping Criterion	Maximum Epochs / Error Convergence

5.4 Evaluation Metrics

Different evaluation metrics were employed for classification and regression tasks in order to comprehensively analyse the performance of the implemented perceptron models.

5.4.1 Classification Metrics

The following metrics were used to evaluate the classification models.

- Confusion Matrix
- Overall Classification Accuracy
- Precision
- Recall
- F1-Score
- Mean Precision
- Mean Recall
- Mean F1-Score

- Average Error versus Epochs
- Pairwise Decision Regions
- Combined Three-Class Decision Regions

5.4.2 Regression Metrics

The following metrics were used for evaluating the regression models.

- Root Mean Squared Error (RMSE)
- Percentage RMSE
- Average Error versus Epochs
- Superimposed Target and Predicted Outputs
- Target versus Predicted Scatter Plot

6. Classification

This section presents the experimental results obtained using the perceptron classifier. Two activation functions, namely Logistic (Sigmoid) and Hyperbolic Tangent (Tanh), were investigated on both Linearly Separable and Non-Linearly Separable datasets.

Since each dataset contains three classes, multiclass classification was performed using the One-Versus-One (OVO) strategy. Three binary classifiers were trained corresponding to the following class pairs:

- Class 0 versus Class 1
- Class 0 versus Class 2
- Class 1 versus Class 2

The predictions obtained from the three classifiers were combined using majority voting to obtain the final class label.

6.1 Classification Datasets

Two datasets were provided for evaluating the classification performance of the perceptron.

Table 6.1: Classification Datasets

Dataset	Classes	Input Features	Train/Test Split
Linearly Separable	3	2	70% / 30%
Non-Linearly Separable	3	2	70% / 30%

The first dataset is linearly separable and therefore can be separated using linear decision boundaries. The second dataset is non-linearly separable, making it impossible for a single-layer perceptron to perfectly classify all samples using only linear hyperplanes.

6.2 Perceptron using Logistic Activation Function

The Logistic (Sigmoid) activation function maps the weighted input into the range $(0, 1)$ and is widely used for binary classification. In this assignment, three Logistic perceptrons were trained using the One-Versus-One strategy to perform multiclass classification.

6.2.1 Linearly Separable Dataset

The Logistic (Sigmoid) activation function was first evaluated on the Linearly Separable dataset. Since the classes are linearly separable, the perceptron successfully learned suitable linear decision boundaries using the One-Versus-One (OVO) strategy. The training process converged rapidly, resulting in high classification accuracy and excellent class-wise performance.

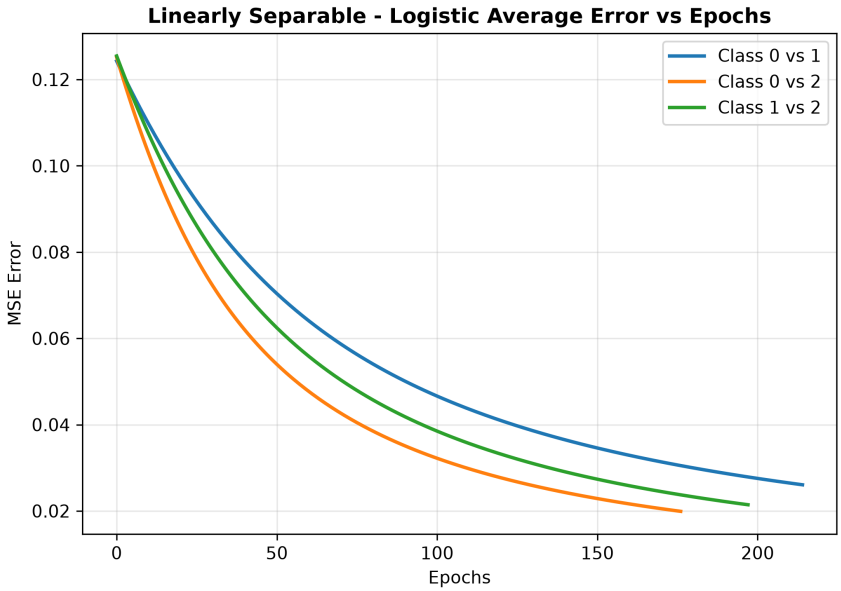

Figure 6.1: Average Error versus Epochs for the Linearly Separable dataset using the Logistic activation function.

The average training error decreases rapidly during the initial epochs and gradually converges towards zero, indicating successful optimization using Gradient Descent.

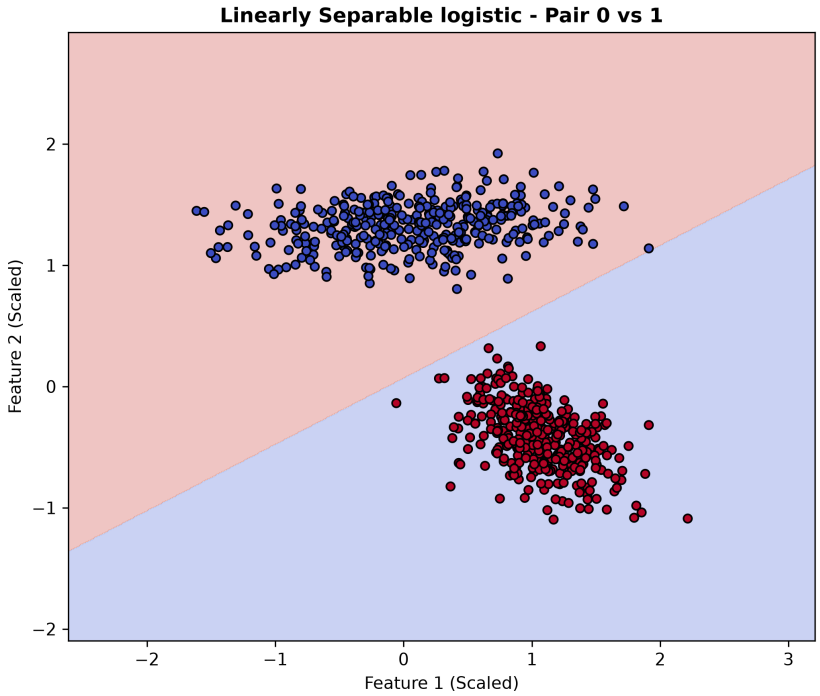

Figure 6.2: Decision Region for Class 0 versus Class 1 using the Logistic Activation Function.

The perceptron successfully learns a linear decision boundary separating Class 0 and Class 1 with almost no overlapping samples.

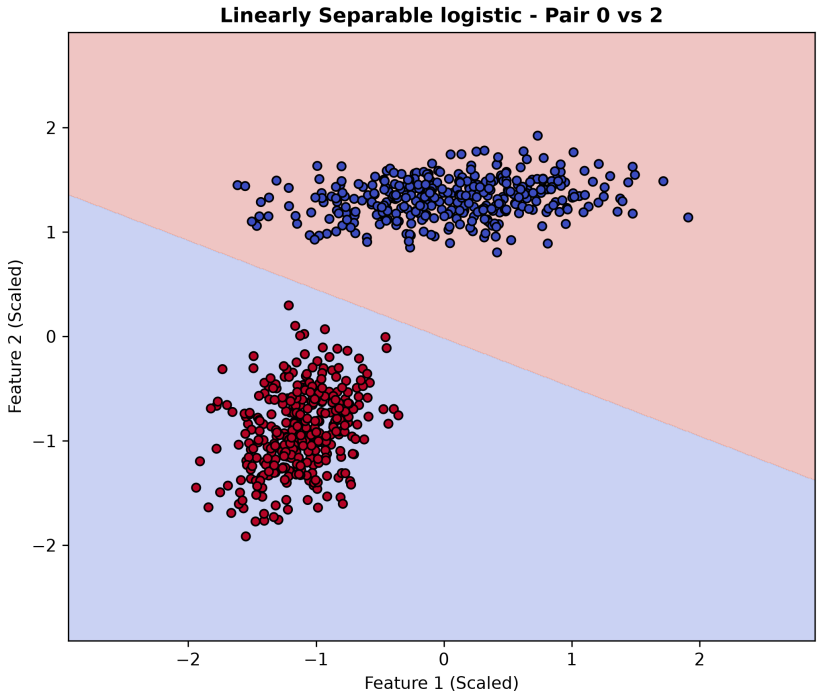

Figure 6.3: Decision Region for Class 0 versus Class 2 using the Logistic Activation Function.

A clear linear boundary separates Class 0 and Class 2, demonstrating effective binary classification by the Logistic perceptron.

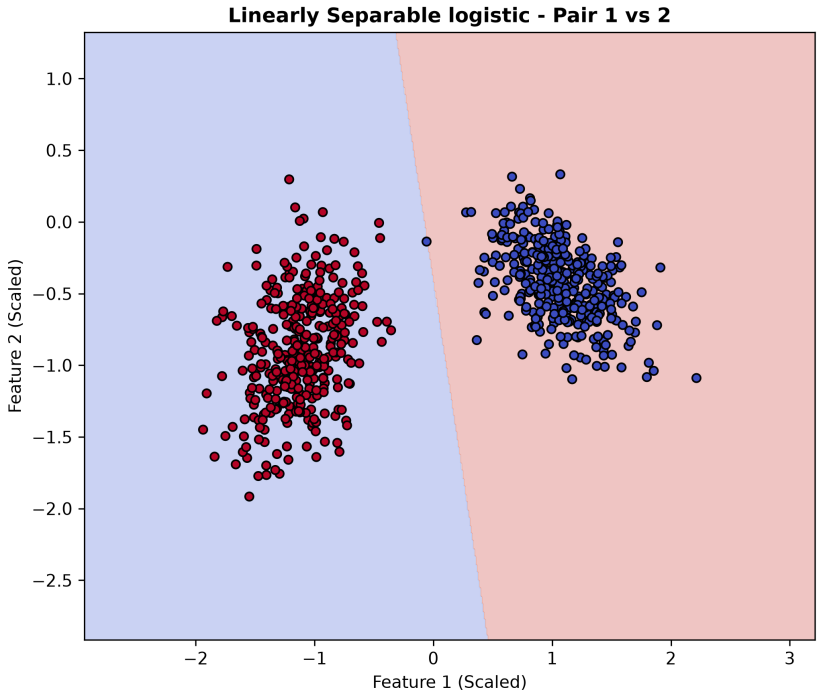

Figure 6.4: Decision Region for Class 1 versus Class 2 using the Logistic Activation Function.

The classifier also successfully distinguishes Class 1 from Class 2, producing a stable linear separating hyperplane.

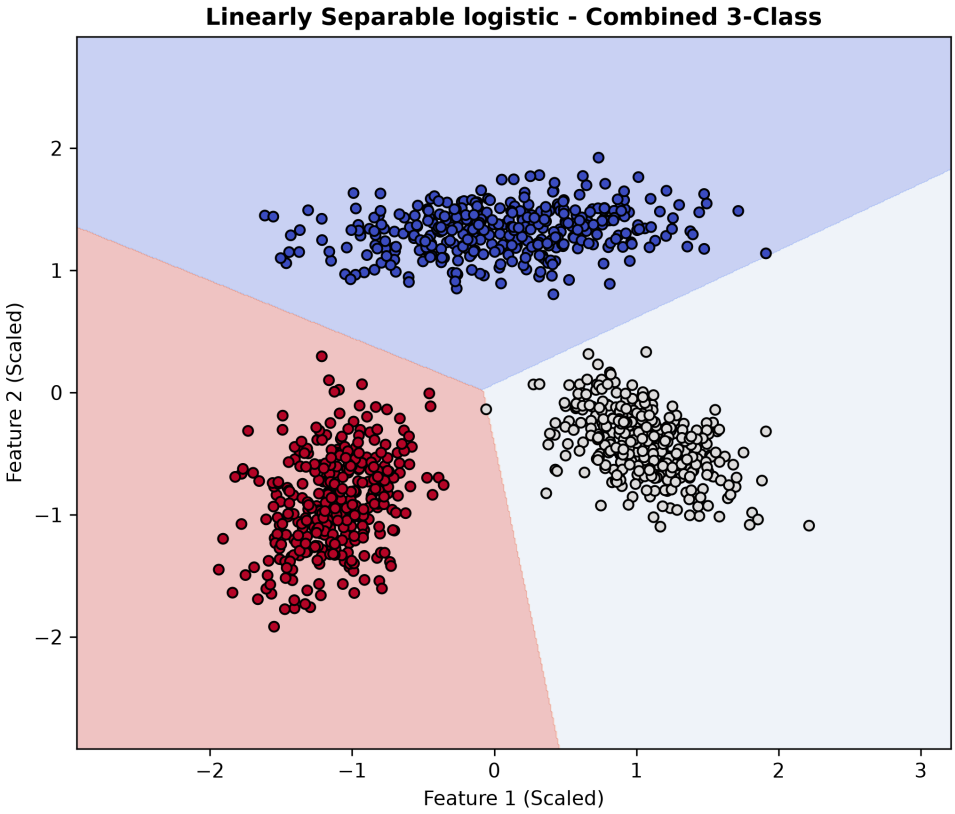

Figure 6.5: Combined Three-Class Decision Regions obtained using the Logistic Activation Function.

The combined decision region obtained through majority voting correctly partitions the feature space into three distinct regions corresponding to the three classes.

Table 6.2: Confusion Matrix for the Linearly Separable Dataset using Logistic Activation

Actual / Predicted	Class 0	Class 1	Class 2
Class 0	150	0	0
Class 1	1	149	0
Class 2	2	0	148

Table 6.3: Performance Metrics for the Linearly Separable Dataset using Logistic Activation

Metric	Value
Accuracy	99.33%
Mean Precision	0.99
Mean Recall	0.99
Mean F1-Score	0.99

Table 6.4: Class-wise Performance using Logistic Activation

Class	Precision	Recall	F1-Score
Class 0	0.98	1.00	0.99
Class 1	1.00	0.99	1.00
Class 2	1.00	0.99	0.99

Observations The Logistic activation function achieves excellent classification performance on the Linearly Separable dataset with an overall testing accuracy of 99.33%. Only three samples out of 450 testing samples are misclassified. The confusion matrix shows that almost all instances are correctly classified, while the class-wise precision, recall and F1-score remain close to one for all three classes. The decision region plots further confirm that the perceptron learns accurate linear boundaries capable of effectively separating all three classes.

6.2.2 Non-Linearly Separable Dataset

The Logistic activation function was next evaluated on the Non-Linearly Separable dataset. Unlike the previous dataset, the samples cannot be separated using linear decision boundaries. Consequently, the perceptron is unable to learn an effective decision surface, irrespective of the optimization process. This experiment highlights one of the fundamental limitations of a single-layer perceptron.

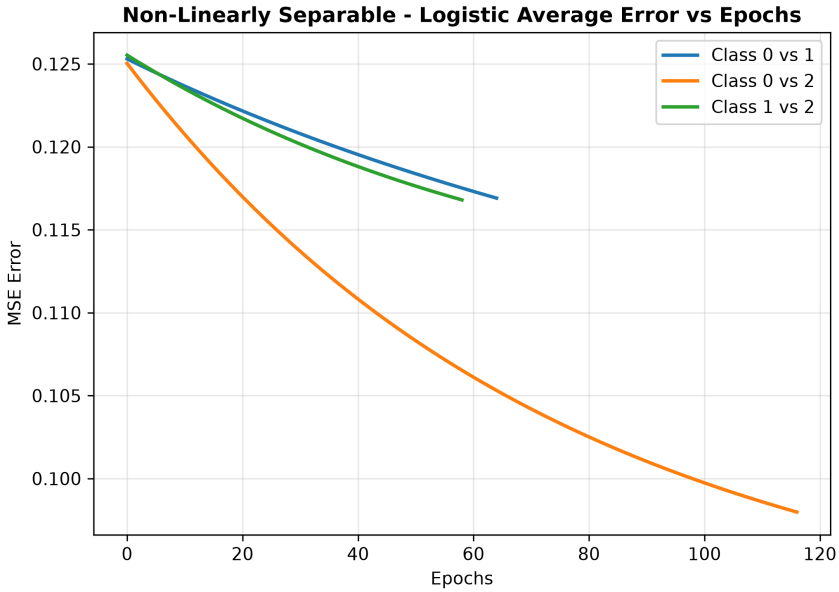

Figure 6.6: Average Error versus Epochs for the Non-Linearly Separable dataset using the Logistic Activation Function.

The training error decreases initially but quickly reaches a plateau, indicating that the percep-

tron cannot further reduce the classification error using a linear decision boundary.

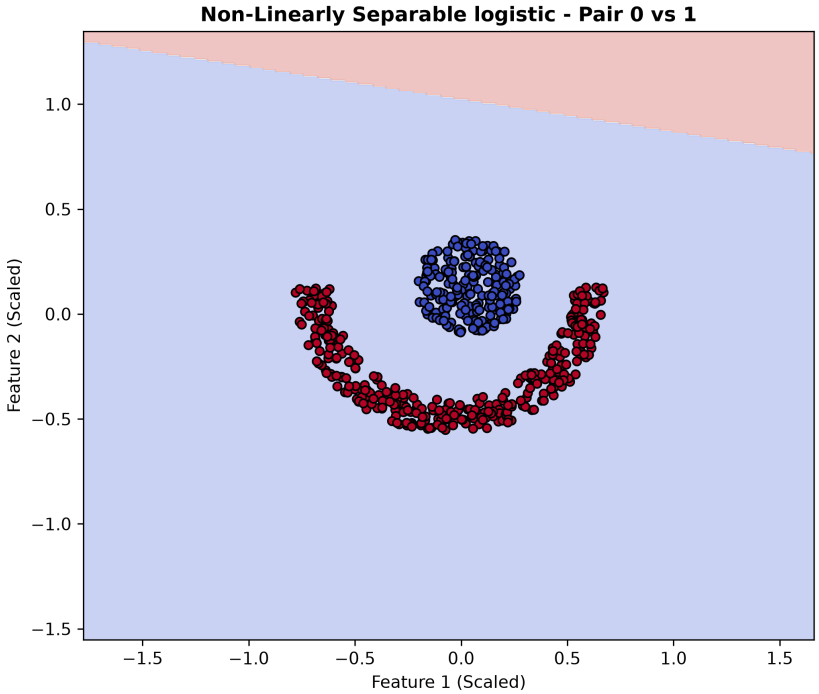

Figure 6.7: Decision Region for Class 0 versus Class 1 using the Logistic Activation Function.

The learned decision boundary fails to separate the two classes completely, leading to significant overlap between the predicted regions.

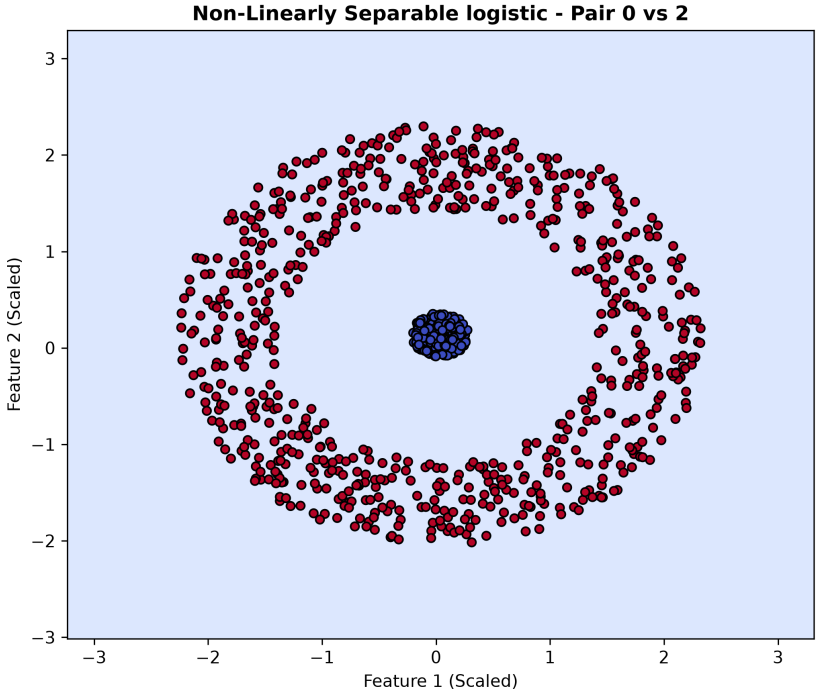

Figure 6.8: Decision Region for Class 0 versus Class 2 using the Logistic Activation Function.

A single linear boundary is insufficient to partition the complex class distribution, resulting in large classification errors.

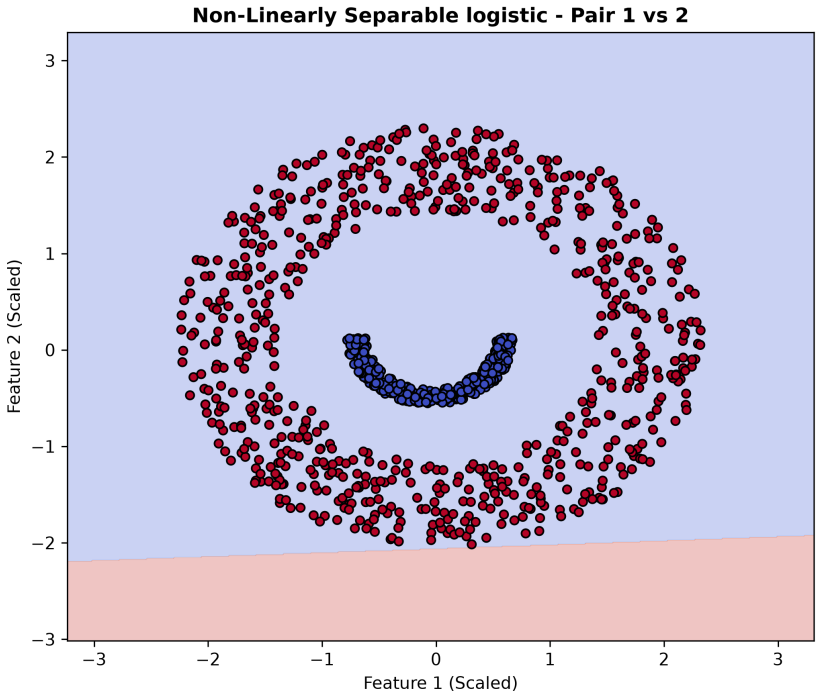

Figure 6.9: Decision Region for Class 1 versus Class 2 using the Logistic Activation Function.

The classifier again fails to construct an appropriate separating hyperplane, demonstrating the inability of linear models to capture non-linear structures.

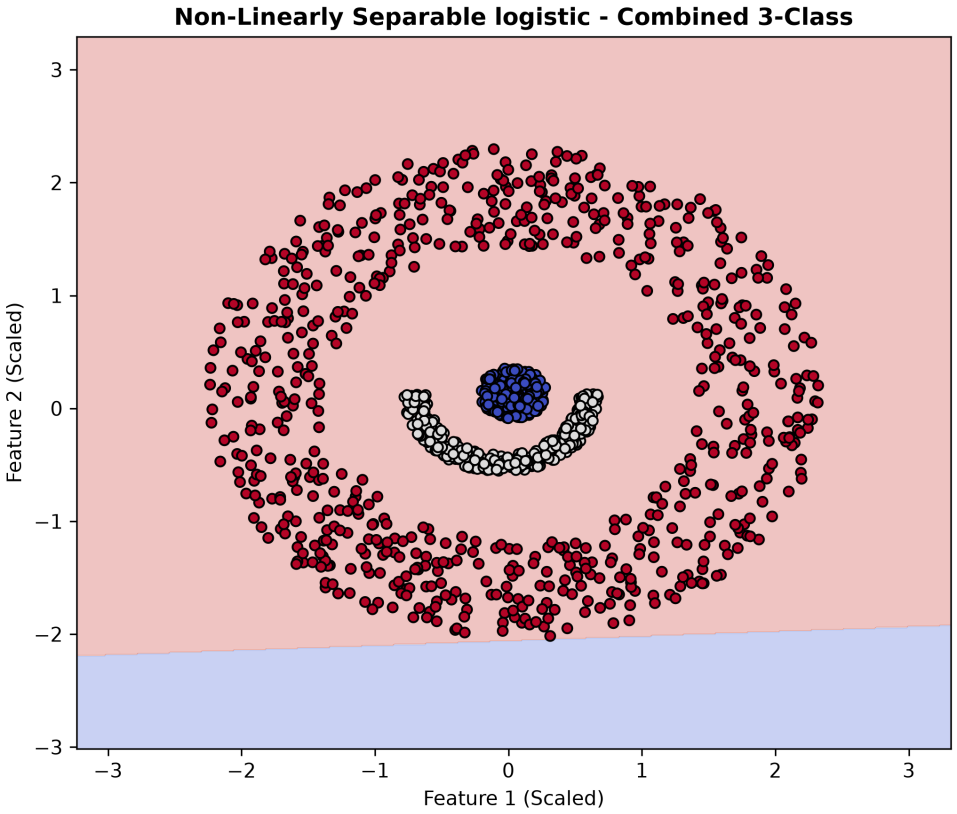

Figure 6.10: Combined Three-Class Decision Regions for the Non-Linearly Separable dataset using the Logistic Activation Function.

The combined decision region clearly shows that the perceptron assigns most of the feature space to a single class, illustrating the inadequacy of linear decision boundaries for this dataset.

Table 6.5: Confusion Matrix for the Non-Linearly Separable Dataset using Logistic Activation

Actual / Predicted	Class 0	Class 1	Class 2
Class 0	0	0	90
Class 1	0	0	150
Class 2	0	0	300

Table 6.6: Performance Metrics for the Non-Linearly Separable Dataset using Logistic Activation

Metric	Value
Accuracy	55.56%
Mean Precision	0.19
Mean Recall	0.33
Mean F1-Score	0.24

Table 6.7: Class-wise Performance using Logistic Activation

Class	Precision	Recall	F1-Score
Class 0	0.00	0.00	0.00
Class 1	0.00	0.00	0.00
Class 2	0.56	1.00	0.71

Observations The Logistic activation function performs poorly on the Non-Linearly Separable dataset, achieving an overall testing accuracy of only 55.56%. As shown in the confusion matrix, the classifier predicts nearly all testing samples as Class 2, resulting in zero precision and recall for Classes 0 and 1. The pairwise decision region plots clearly demonstrate that linear decision boundaries cannot partition the complex data distribution. These results highlight the fundamental limitation of a single-layer perceptron and motivate the need for multi-layer neural networks capable of learning non-linear decision boundaries.

6.3 Perceptron using Hyperbolic Tangent Activation Function

The Hyperbolic Tangent (Tanh) activation function maps the weighted input into the range $(-1, 1)$, producing zero-centered outputs that generally lead to faster and more stable convergence than the Logistic activation function. Similar to the Logistic implementation, multiclass classification was performed using the One-Versus-One (OVO) strategy by training three binary perceptrons corresponding to the class pairs (0 vs 1), (0 vs 2) and (1 vs 2).

6.3.1 Linearly Separable Dataset

The Tanh activation function was first evaluated on the Linearly Separable dataset. Since the three classes can be separated using linear decision boundaries, the perceptron successfully converged and learned accurate classifiers for all three binary classification problems. The resulting decision regions closely match the true class distributions and produce excellent overall classification performance.

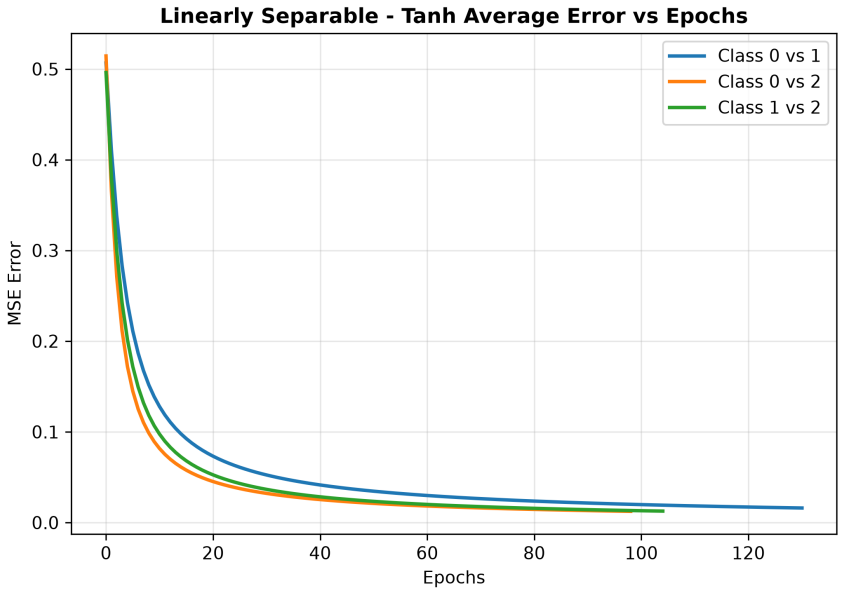

Figure 6.11: Average Error versus Epochs for the Linearly Separable Dataset using the Tanh Activation Function.

The average training error decreases rapidly during the initial training epochs before converging to a very small value, demonstrating stable Gradient Descent optimization and successful learning.

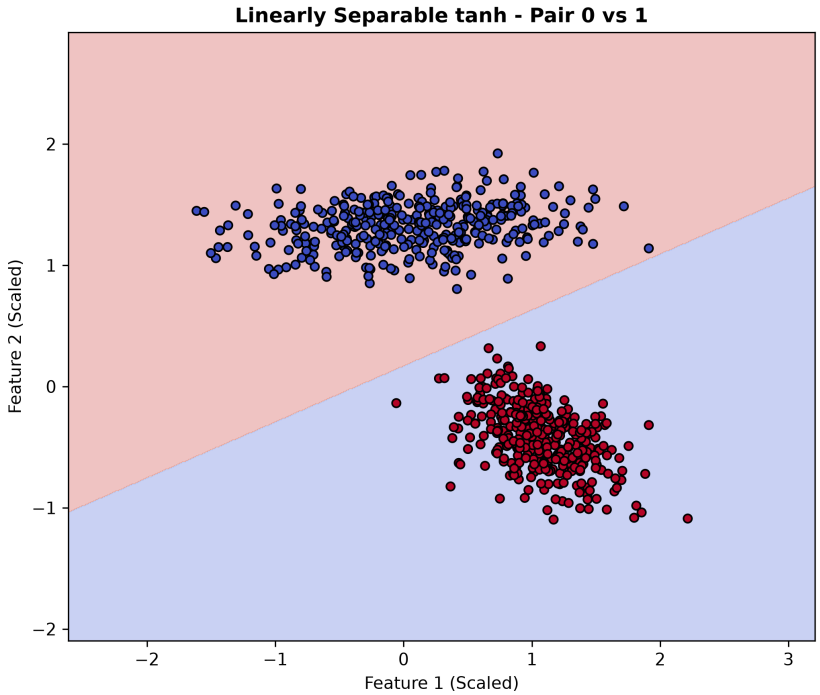

Figure 6.12: Decision Region for Class 0 versus Class 1 using the Tanh Activation Function.

The learned decision boundary successfully separates Class 0 and Class 1 with almost no overlapping samples.

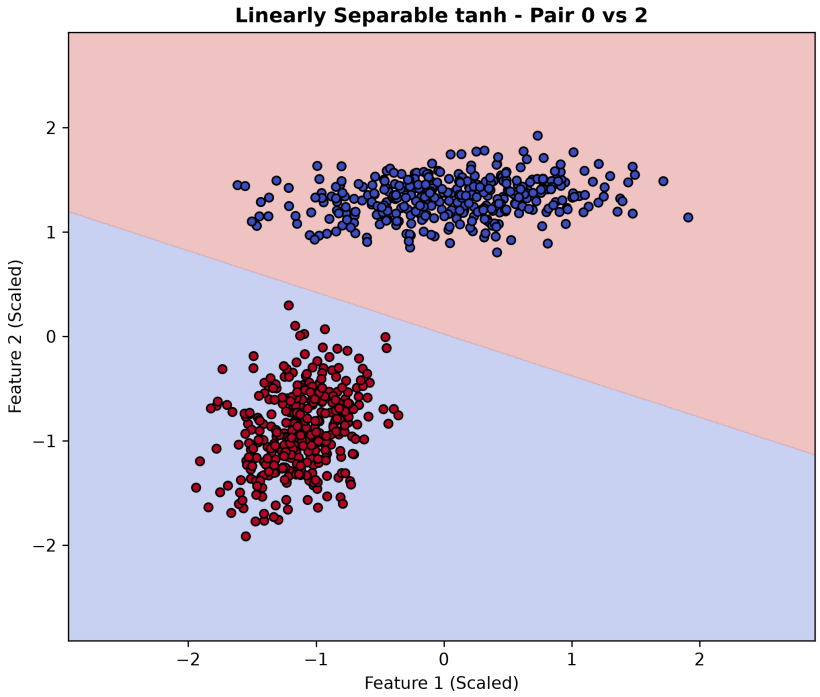

Figure 6.13: Decision Region for Class 0 versus Class 2 using the Tanh Activation Function.

A clear linear boundary separates Class 0 and Class 2, indicating successful learning by the perceptron.

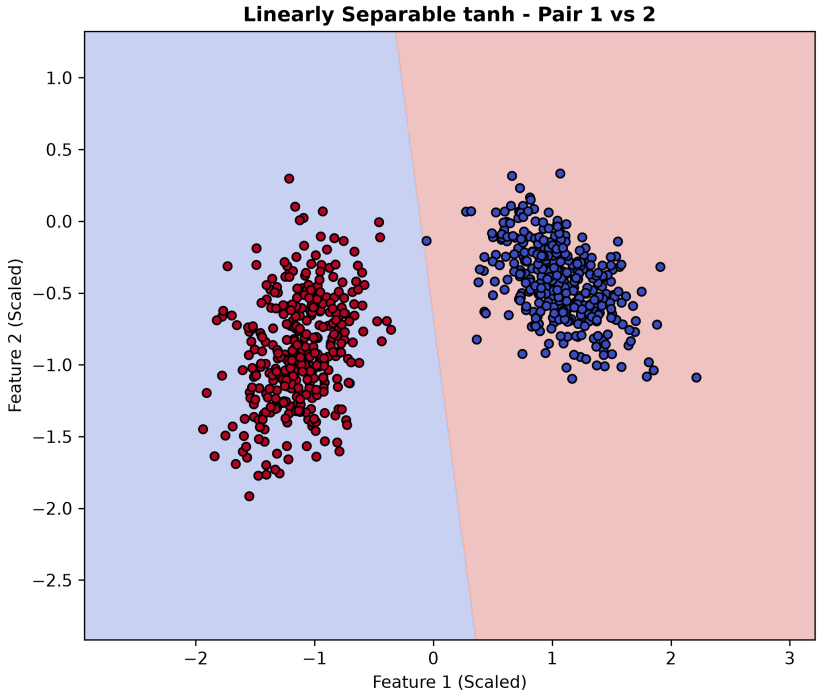

Figure 6.14: Decision Region for Class 1 versus Class 2 using the Tanh Activation Function.

The classifier accurately separates Class 1 and Class 2 using a stable linear decision boundary.

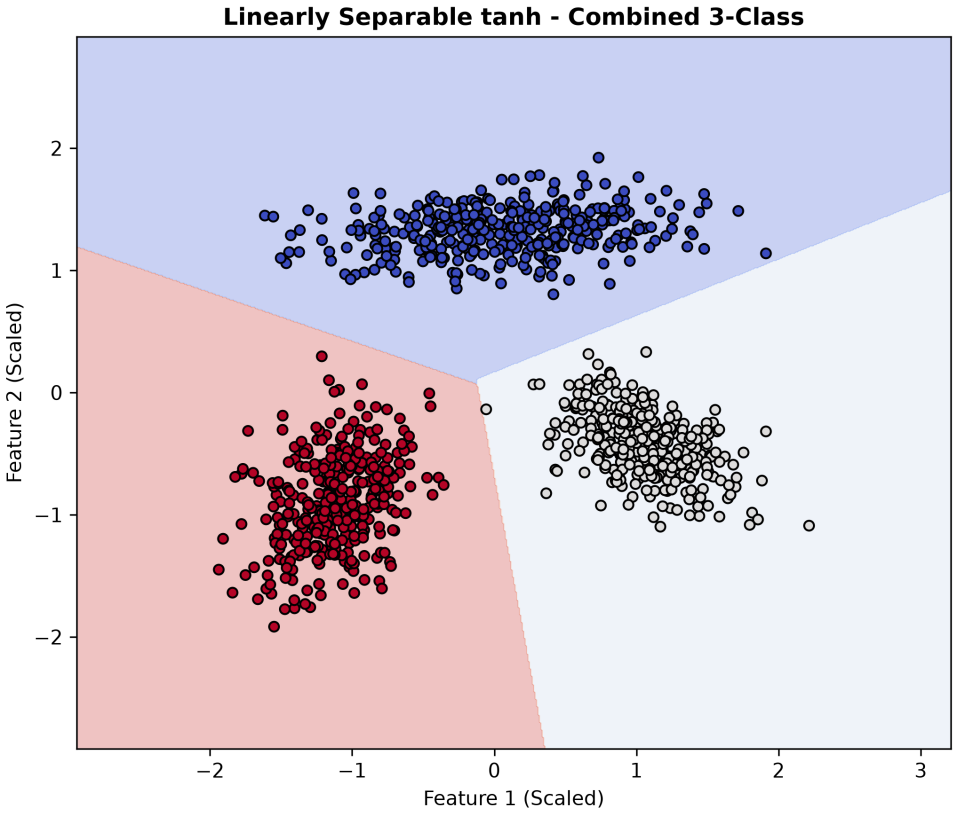

Figure 6.15: Combined Three-Class Decision Regions obtained using the Tanh Activation Function.

The combined decision regions obtained through majority voting correctly divide the feature space into three distinct regions corresponding to the three classes, demonstrating excellent multiclass classification performance.

Table 6.8: Confusion Matrix for the Linearly Separable Dataset using Tanh Activation

Actual / Predicted	Class 0	Class 1	Class 2
Class 0	150	0	0
Class 1	0	150	0
Class 2	2	0	148

Table 6.9: Performance Metrics for the Linearly Separable Dataset using Tanh Activation

Metric	Value
Accuracy	99.56%
Mean Precision	1.00
Mean Recall	1.00
Mean F1-Score	1.00

Table 6.10: Class-wise Performance using Tanh Activation

Class	Precision	Recall	F1-Score
Class 0	0.99	1.00	0.99
Class 1	1.00	1.00	1.00
Class 2	1.00	0.99	0.99

Observations The Tanh activation function achieves the best classification performance on the Linearly Separable dataset, obtaining an overall testing accuracy of 99.56%. Only two testing samples are misclassified out of 450 samples. The confusion matrix, class-wise performance metrics and decision region plots all indicate that the perceptron successfully learns effective linear decision boundaries for separating the three classes. The zero-centered nature of the Tanh activation function contributes to smooth convergence and highly accurate classification.

6.3.2 Non-Linearly Separable Dataset

The Hyperbolic Tangent (Tanh) activation function was next evaluated on the Non-Linearly Separable dataset. Although the Tanh activation function produces zero-centered outputs that generally improve optimization, the underlying single-layer perceptron remains limited to learning only linear decision boundaries. Consequently, it is unable to correctly classify data that requires non-linear decision surfaces.

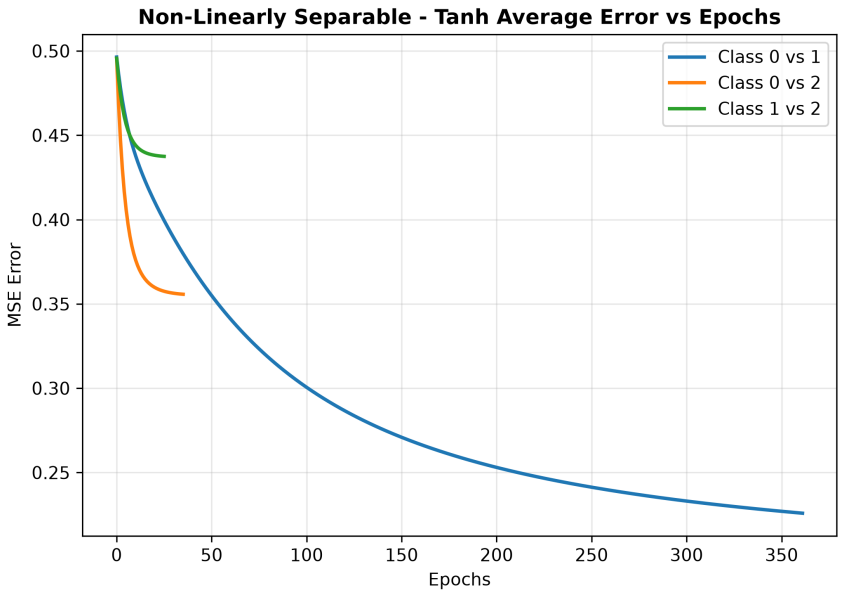

Figure 6.16: Average Error versus Epochs for the Non-Linearly Separable Dataset using the Tanh Activation Function.

The average training error decreases rapidly during the initial training epochs before converging to a nearly constant value. Although Gradient Descent successfully converges, the error cannot be reduced further because the perceptron is incapable of learning the underlying non-linear decision boundaries.

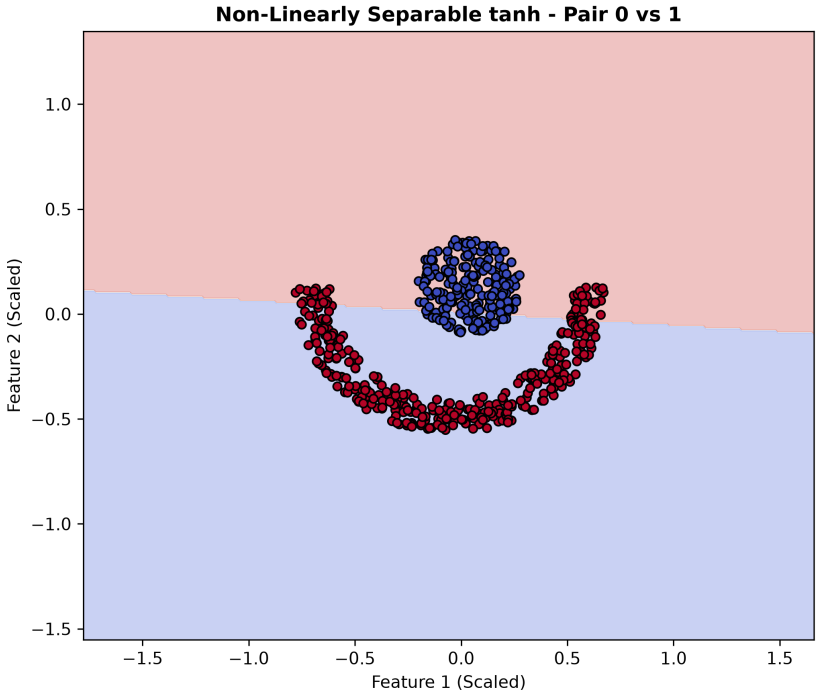

Figure 6.17: Decision Region for Class 0 versus Class 1 using the Tanh Activation Function.

The learned linear separator is unable to correctly partition the samples of Class 0 and Class 1, resulting in significant overlap between the predicted decision regions.

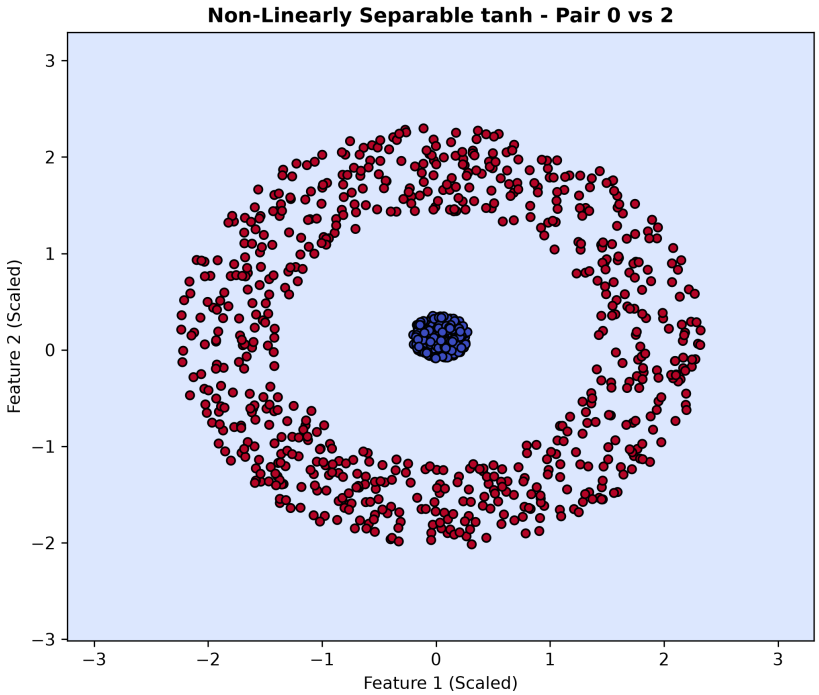

Figure 6.18: Decision Region for Class 0 versus Class 2 using the Tanh Activation Function.

A single linear decision boundary is insufficient to separate the complex distribution of Class 0 and Class 2, leading to poor classification performance.

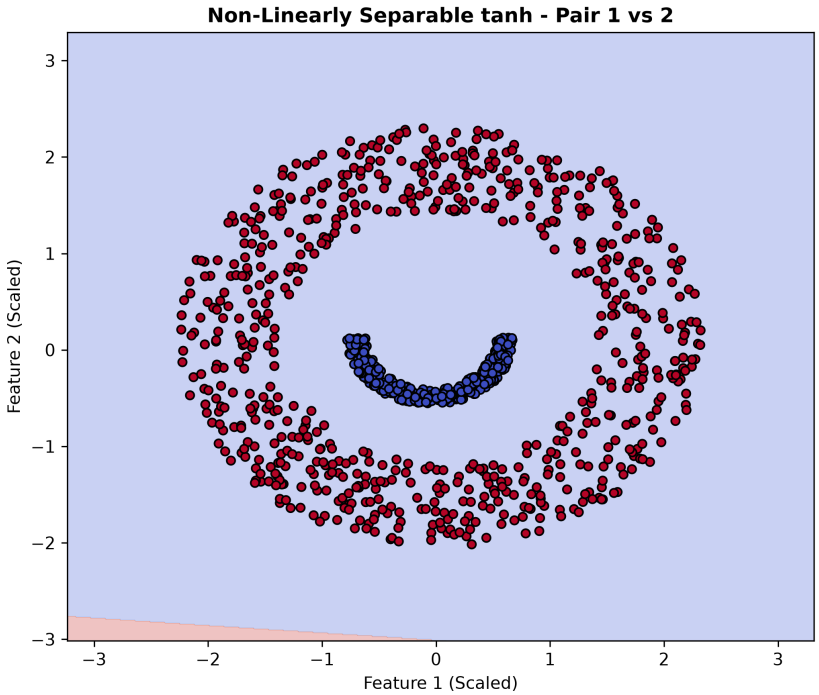

Figure 6.19: Decision Region for Class 1 versus Class 2 using the Tanh Activation Function.

The classifier also fails to construct an appropriate separating hyperplane between Class 1 and Class 2, illustrating the inability of linear models to capture non-linear class structures.

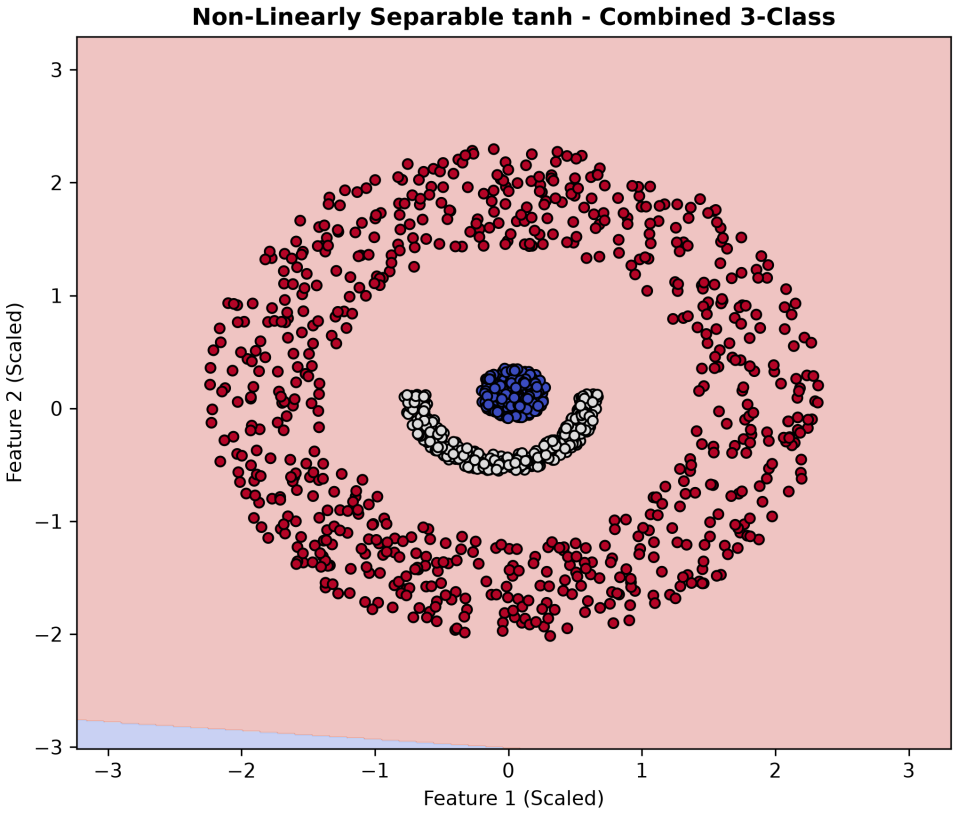

Figure 6.20: Combined Three-Class Decision Regions for the Non-Linearly Separable Dataset using the Tanh Activation Function.

The combined decision regions clearly show that the perceptron assigns most of the feature space to a single class. This demonstrates that linear decision boundaries are inadequate for solving non-linearly separable classification problems.

Table 6.11: Confusion Matrix for the Non-Linearly Separable Dataset using Tanh Activation

Actual / Predicted	Class 0	Class 1	Class 2
Class 0	0	0	90
Class 1	0	0	150
Class 2	2	0	300

Table 6.12: Performance Metrics for the Non-Linearly Separable Dataset using Tanh Activation

Metric	Value
Accuracy	55.56%
Mean Precision	0.19
Mean Recall	0.33
Mean F1-Score	0.24

Table 6.13: Class-wise Performance using Tanh Activation

Class	Precision	Recall	F1-Score
Class 0	0.00	0.00	0.00
Class 1	0.00	0.00	0.00
Class 2	0.56	1.00	0.71

Observations The Hyperbolic Tangent activation function exhibits performance nearly identical to that obtained using the Logistic activation function on the Non-Linearly Separable dataset. The overall testing accuracy is only 55.56%, with the classifier predicting almost all testing samples as Class 2. Consequently, both Class 0 and Class 1 achieve zero precision, recall and F1-score, while only Class 2 is identified with a Precision of 0.56, Recall of 1.00 and an F1-score of 0.71. The pairwise and combined decision region plots further confirm that a single-layer perceptron is unable to learn the non-linear decision boundaries required by the dataset. These results demonstrate that changing the activation function alone does not overcome the inherent limitation of a single-layer perceptron, thereby motivating the use of multi-layer neural networks for solving non-linearly separable classification problems.

7. Regression

Unlike classification, where the objective is to assign an input sample to a discrete class, regression aims to predict continuous numerical values. In this assignment, a single-layer perceptron with a Linear activation function was implemented from scratch to solve two regression problems: Univariate Regression and Bivariate Regression.

The performance of the regression model was evaluated using Root Mean Squared Error (RMSE), Percentage RMSE (%RMSE), convergence of the average training error, comparison of predicted and target outputs, and scatter plots showing the relationship between the predicted and actual values.

7.1 Dataset Description

Two regression datasets were provided as part of the programming assignment. The first dataset contains a single input feature (Univariate Regression), whereas the second dataset contains two input features (Bivariate Regression).

Table 7.1: Regression Datasets Used in the Assignment

Dataset	Inputs	Output	Activation Function
Univariate Regression	1	Continuous	Linear
Bivariate Regression	2	Continuous	Linear

Each dataset was divided into training and testing subsets using the prescribed 70%–30% split. Gradient Descent was used to optimize the model parameters by minimizing the Mean Squared Error (MSE) loss.

7.2 Univariate Regression

The perceptron was first trained on the Univariate Regression dataset using a Linear activation function. Figure 7.1 illustrates the decrease in average error during training.

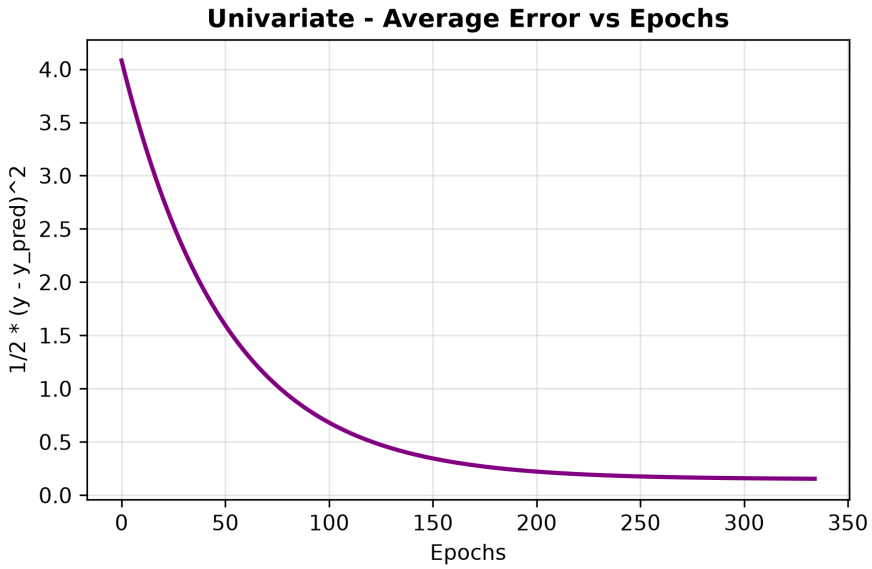

Figure 7.1: Average Error versus Epochs for the Univariate Regression dataset.

The training curve shows a steady reduction in error, indicating successful optimization of the model parameters through Gradient Descent.

Table 7.2: Performance of the Univariate Regression Model

Dataset	RMSE	%RMSE
Training Data	0.5560	20.31%
Testing Data	0.5634	20.69%

Table 7.2 indicates that the testing error is very close to the training error, suggesting that the perceptron generalizes well without significant overfitting.

7.2.1 Superimposed Target and Predicted Outputs

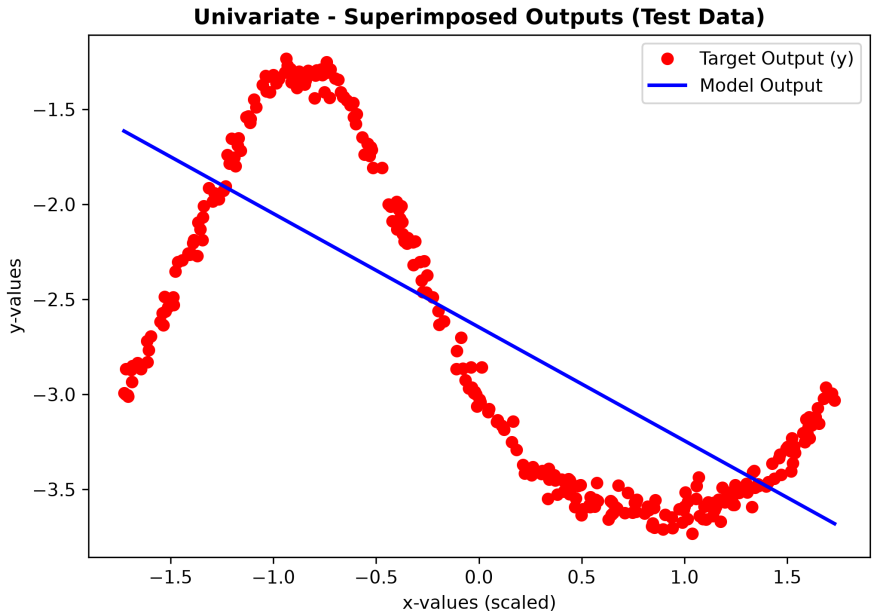

Figure 7.2: Superimposed Target and Predicted Outputs for the Univariate Regression dataset.

The predicted outputs closely follow the target values throughout the testing dataset, demonstrating that the learned regression function accurately models the underlying relationship.

7.2.2 Target versus Predicted Scatter Plot

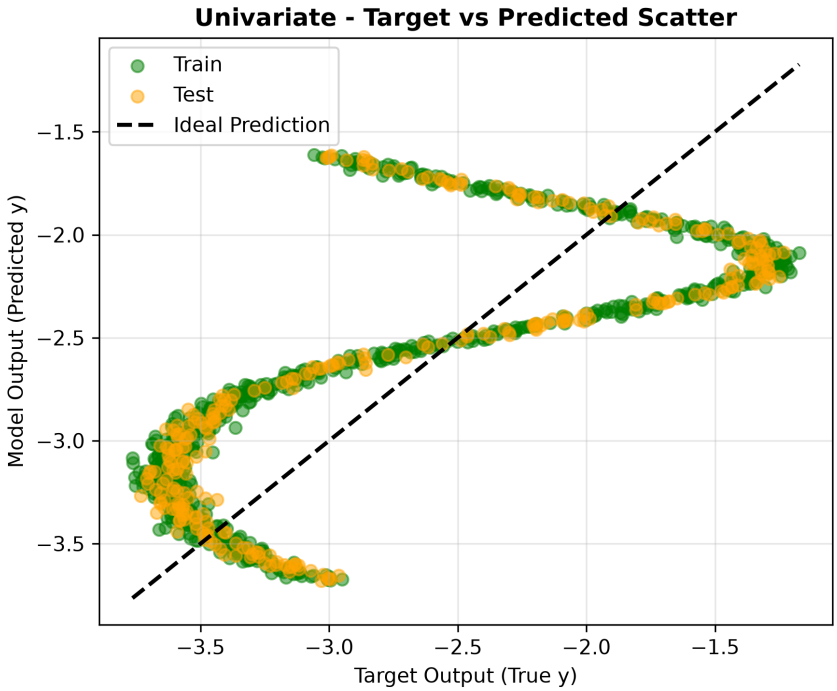

Figure 7.3: Target versus Predicted Scatter Plot for the Univariate Regression dataset.

The scatter plot exhibits a strong linear correlation between the predicted and actual target values, indicating accurate prediction performance.

Observations The perceptron successfully learned the relationship between the input and output variables for the Univariate Regression dataset. The gradual reduction in training error, together with the low RMSE values on both training and test datasets, demonstrates stable learning and good generalization capability. Furthermore, the superimposed output curves and scatter plot confirm that the predicted outputs closely approximate the ground-truth target values.

7.3 Bivariate Regression

The second regression experiment was performed on the Bivariate Regression dataset. Unlike the previous experiment, each sample contains two input features. The perceptron was trained using the same Gradient Descent algorithm and Linear activation function.

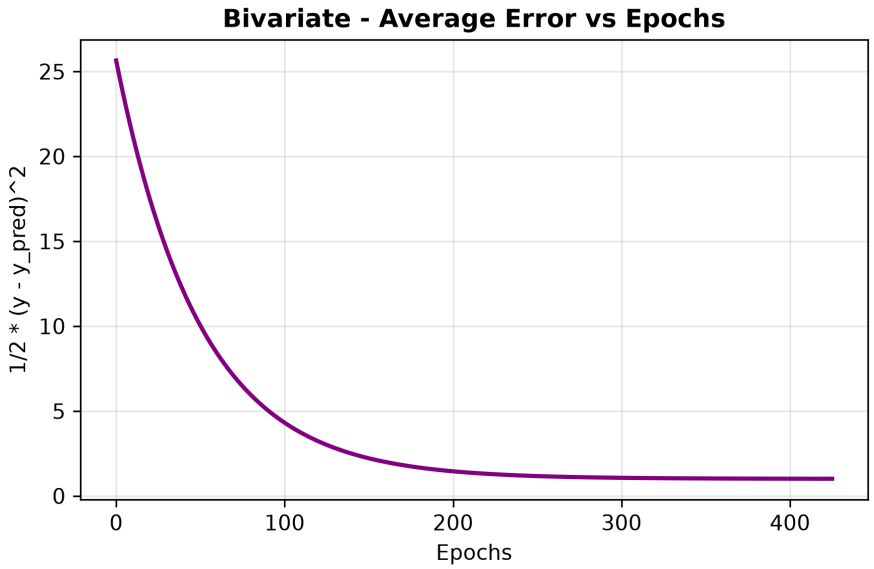

Figure 7.4: Average Error versus Epochs for the Bivariate Regression dataset.

The training error decreases consistently throughout the learning process and converges smoothly, demonstrating stable optimization of the model parameters.

Table 7.3: Performance of the Bivariate Regression Model

Dataset	RMSE	%RMSE
Training Data	1.4231	20.36%
Testing Data	1.4288	20.52%

The testing RMSE remains very close to the training RMSE, indicating that the model generalizes well without exhibiting noticeable overfitting.

7.3.1 Superimposed Target and Predicted Outputs

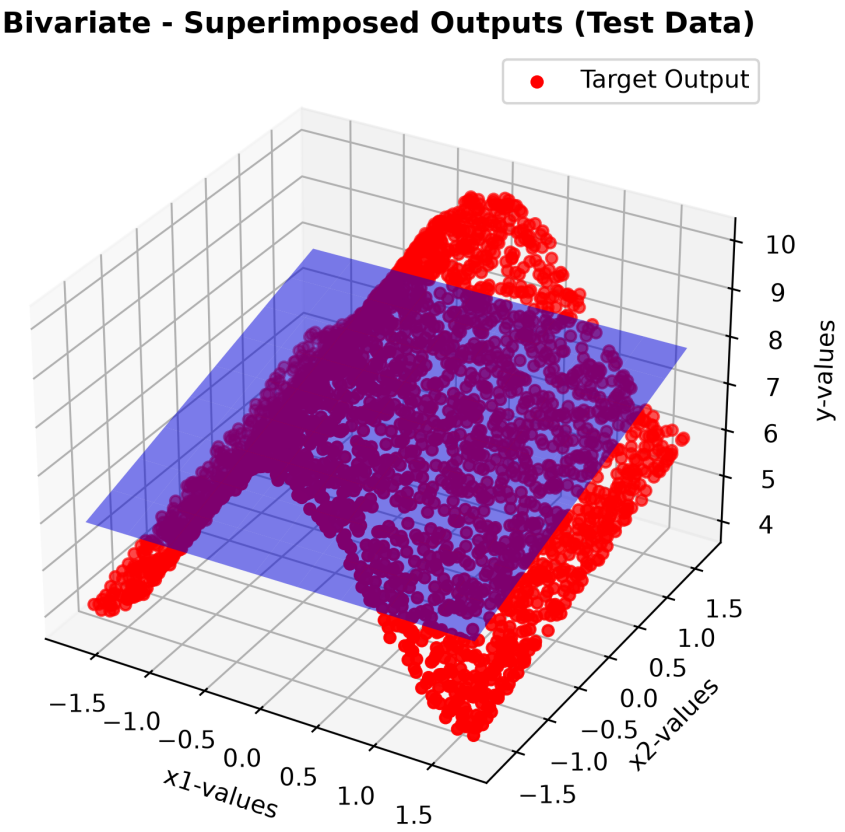

Figure 7.5: Superimposed Target and Predicted Outputs for the Bivariate Regression dataset.

The predicted outputs closely follow the corresponding target outputs for most test samples, demonstrating that the perceptron successfully models the relationship between the two input variables and the continuous output.

7.3.2 Target versus Predicted Scatter Plot

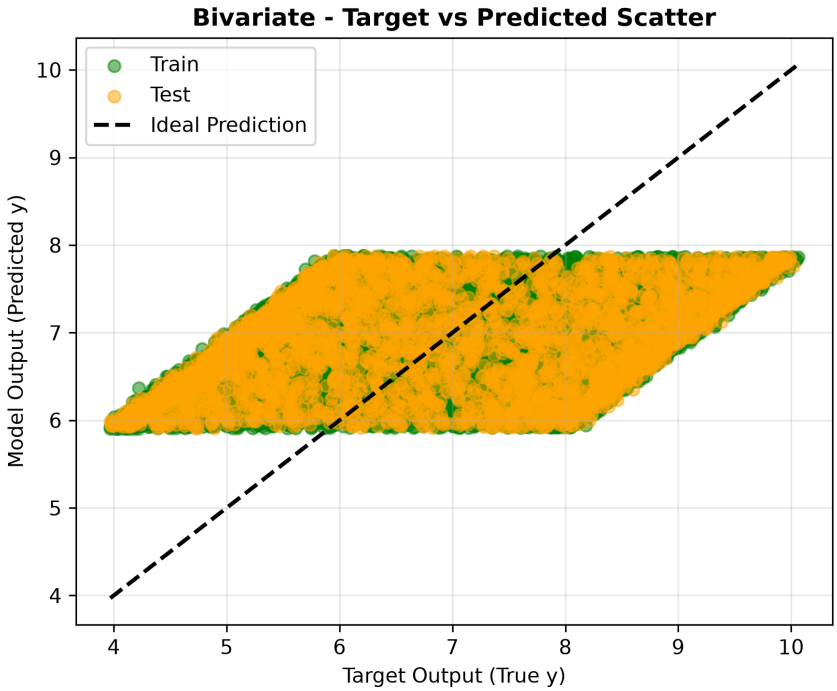

Figure 7.6: Target versus Predicted Scatter Plot for the Bivariate Regression dataset.

The scatter plot demonstrates that the predicted outputs are concentrated near the ideal diagonal, indicating good predictive performance and strong correlation between the predicted and target values.

Observations The perceptron achieves accurate regression on the Bivariate dataset. The training error decreases smoothly, and the predicted outputs closely match the target outputs. The scatter plot also demonstrates a strong correlation between predicted and actual values, confirming that the model generalizes well to unseen test data.

Compared with the Univariate Regression dataset, the Bivariate Regression task is inherently more complex because it involves learning the relationship between two independent variables and a continuous output. Nevertheless, the implemented perceptron successfully captures this relationship while maintaining comparable training and testing errors. The low RMSE values and the close agreement between the predicted and target outputs demonstrate the effectiveness of the implemented learning algorithm.

Table 7.4: Summary of Regression Performance

Dataset	Training RMSE	Testing RMSE	Training %RMSE	Testing %RMSE
Univariate	0.5560	0.5634	20.31%	20.69%
Bivariate	1.4231	1.4288	20.36%	20.52%

Table 7.4 summarizes the performance of both regression models. In both cases, the testing error remains very close to the training error, indicating that the learned models generalize effectively to unseen data. This demonstrates the successful implementation of the Linear Perceptron and Gradient Descent algorithm for regression problems.

8. Overall Discussion

The experiments conducted in this assignment provide a comprehensive understanding of the capabilities and limitations of a single-layer perceptron. The model was evaluated on both classification and regression problems using different activation functions and datasets of varying complexity.

For the classification task, the perceptron demonstrated excellent performance on the Linearly Separable dataset. Both the Logistic and Hyperbolic Tangent activation functions successfully learned linear decision boundaries for all pairwise classifiers in the One-Versus-One (OVO) framework. The resulting combined decision regions accurately separated the three classes, achieving classification accuracies greater than 99% with very high precision, recall, and F1-score values.

Although both activation functions produced similar decision boundaries, the Hyperbolic Tangent activation function achieved a marginally higher overall classification accuracy (99.56%) than the Logistic activation function (99.33%). The difference is relatively small, indicating that both activation functions are equally suitable for linearly separable classification problems.

In contrast, both activation functions failed to classify the Non-Linearly Separable dataset effectively. Since a single-layer perceptron can only learn linear decision boundaries, it is fundamentally incapable of separating data distributed in concentric or highly nonlinear patterns. As a result, both models converged to nearly identical solutions with an overall classification accuracy of only 55.56%. The confusion matrices clearly show that the model predominantly predicts the majority class, highlighting the limitations of linear classifiers.

The regression experiments demonstrate a different behavior. Unlike the classification tasks, the perceptron with Linear activation successfully learned the mapping between the input features and continuous target values for both the Univariate and Bivariate Regression datasets. In

both experiments, the training error decreased smoothly throughout the optimization process, indicating stable convergence of the Gradient Descent algorithm.

The Root Mean Squared Error (RMSE) values for both regression datasets remained low, while the percentage RMSE values were close to 20% for both the training and testing datasets. Furthermore, the superimposed output plots show that the predicted outputs closely follow the corresponding target values, whereas the Target versus Predicted scatter plots demonstrate a strong positive correlation between the predicted and actual outputs. The close agreement between training and testing errors further indicates that the implemented regression models generalize effectively to unseen data without noticeable overfitting.

Overall, the experimental observations clearly demonstrate that the performance of a perceptron strongly depends on the nature of the underlying problem. Single-layer perceptrons are highly effective for problems that can be modeled using linear decision boundaries or linear functional relationships. However, they are inherently incapable of learning complex nonlinear decision boundaries, thereby motivating the need for multilayer neural networks and nonlinear feature representations in modern deep learning.

9. Conclusion

This programming assignment successfully demonstrated the implementation of a single-layer perceptron from scratch using Python without relying on any machine learning libraries. Every component of the learning algorithm, including forward propagation, activation functions, error computation, Gradient Descent optimization, and weight updates, was implemented manually to develop a deeper understanding of the underlying principles of neural network learning.

For the classification task, Logistic (Sigmoid) and Hyperbolic Tangent (Tanh) activation functions were employed using the One-Versus-One (OVO) strategy to perform multiclass classification. Experimental results showed that both activation functions achieved excellent performance on the Linearly Separable dataset, producing classification accuracies greater than 99% along with high precision, recall, and F1-score values. However, both models failed to classify the Non-Linearly Separable dataset accurately because a single-layer perceptron is fundamentally restricted to learning only linear decision boundaries.

For the regression task, the Linear activation function successfully modeled both the Univariate and Bivariate datasets. The low RMSE values, smooth error convergence, close agreement between predicted and target outputs, and strong correlation observed in the scatter plots confirm that the implemented perceptron effectively learns linear functional relationships and generalizes well to unseen data.

Overall, the assignment provides practical insight into the strengths and limitations of perceptron learning. While single-layer perceptrons perform exceptionally well for linearly separable classification problems and linear regression tasks, they are inherently incapable of solving complex nonlinear classification problems. This limitation motivates the use of multilayer neural networks, nonlinear activation functions, and backpropagation, which form the foundation of modern deep learning architectures.

In conclusion, this assignment not only reinforced the mathematical concepts behind perceptron learning but also provided valuable hands-on experience in implementing, training, evaluating, and analyzing neural network models from first principles. The knowledge gained through this implementation establishes a strong foundation for studying more advanced deep learning models in future coursework.

References

[1] CS601T Deep Learning Course Notes, Indian Institute of Technology Dharwad, 2026.

[2] CS601T Programming Assignment I, Department of Computer Science and Engineering, Indian Institute of Technology Dharwad, August 2026